

Unit-4 (2 Marks)

1. What are servlets?

Servlets are small programs that execute on the server side of a web connection. Just as applets dynamically extend the functionality of a web browser, servlets dynamically extend the functionality of a web server

2. List the drawbacks of CGI programs.

- CGI allowed the separate process to read data from the HTTP request and write data to the HTTP response.
- A variety of different languages were used to build CGI programs. These included C, C++, and Perl
- However, CGI suffered serious performance problems.
- It was expensive in terms of processor and memory resources to create a separate process for each client request.
- It was also expensive to open and close database connections for each client request.
- In addition the CGI programs were not platform-independent

3. List the advantages of servlet.

- Servlets offer several advantages in comparison with CGI.
- First, performance is significantly better.
- Servlets execute within the address space of a web server.
- It is not necessary to create a separate process to handle each client request.
- Second, servlets are platform-independent because they are written in Java.
- Third, the Java security manager on the server enforces a set of restrictions to protect the resources on a server machine.
- Finally, the full functionality of the Java class libraries is available to a servlet. It can communicate with applets, databases, or other software via the sockets and RMI mechanisms

4. With syntax write the purpose of `getParameter()` method.

<code>String getParameter(String pname)</code>	Returns the value of the parameter named pname.
---	---

5. What is the purpose of extending the GenericServlet class in the servlet, and what methods does it provide by default?

- The **GenericServlet** class provides implementations of the basic life cycle methods for a servlet.
- GenericServlet implements the *Servlet* and *ServletConfig* interfaces.
- In addition, a method to append a string to the server log file is available.
- The signatures of this method are shown here:
- **void log(String s)** **void log(String s, Throwable e)**
- Here, s is the string to be appended to the log, and e is an exception that occurred

6. Explain the role of the ServletRequest and ServletResponse objects in the service() method

ServletRequest	Used to read data from a client request.
ServletResponse	Used to write data to a client response.
void service(ServletRequest req, ServletResponse res) throws ServletException, IOException	Called to process a request from a client. The request from the client can be read from req. The response to the client can be written to res. An exception is generated if a servlet or IO problem occurs.

7. What is ServletConfig Interface? Mention any 2 methods

The ServletConfig interface allows a servlet to obtain configuration data when it is loaded. The methods declared by this interface are summarized here:

Method	Description
ServletContext getServletContext()	Returns the context for this servlet.
String getInitParameter(String param)	Returns the value of the initialization parameter named param.

Enumeration getInitParameterNames()	Returns an enumeration of all initialization parameter names.
String getServletName()	Returns the name of the invoking servlet.

8. What is the purpose and significance of the getWriter() method in the context of generating an HTTP response in a servlet?

PrintWriter getWriter() throws IOException	Returns a PrintWriter that can be used to write character data to the response. An <i>IllegalStateException</i> is thrown if <i>getOutputStream()</i> has already been invoked for this request.
--	--

PrintWriter pw = response.getWriter();

9. _____ Servlet _____ interface declares life cycle methods for a servlet and _____ ServletConfig _____ interface allows servlets to get initialization parameters.

10. What is the role of the getServletConfig() and getServletInfo() methods in the Servlet interface

- The **getServletConfig()** method is called by the servlet to obtain initialization parameters.
- A servlet developer overrides the **getServletInfo()** method to provide a string with useful information (for example, author, version, date, copyright). This method is also invoked by the server

11. Differentiate between the getInitParameter(String param) and getInitParameterNames() methods in the ServletConfig interface

String getInitParameter(String param)	Returns the value of the initialization parameter named param.
Enumeration getInitParameterNames()	Returns an enumeration of all initialization parameter names.

12. What the purpose and usage of the `getAttribute(String attr)` and `setAttribute(String attr, Object val)` methods in the `ServletContext` interface

Object <code>getAttribute(String attr)</code>	Returns the value of the server attribute named <code>attr</code> .
void <code>setAttribute(String attr, Object val)</code>	Sets the attribute specified by <code>attr</code> to the value passed in <code>val</code> .

13. What is the purpose of the `getParameterNames()` and `getParameterValues(String name)` methods in the `ServletRequest` interface.

Enumeration <code>getParameterNames()</code>	Returns an enumeration of the parameter names for this request.
String[] <code>getParameterValues(String name)</code>	Returns an array containing values associated with the parameter specified by <code>name</code> .

14. What is the usage of `readLine(byte[] buffer, int offset, int size)` method in the `ServletInputStream` class?

Method is provided to read bytes from the stream
`int readLine(byte[] buffer, int offset, int size)` throws `IOException`

Here, *buffer* is the array into which *size* bytes are placed starting at *offset*. The method returns the actual number of bytes read or -1 if an end-of-stream condition is encountered.

15. What's the specific purpose of `ServletOutputStream` and `ServletInputStream`?

The `ServletInputStream` class extends `InputStream`. It is implemented by the servlet container and provides an input stream that a servlet developer can use to read the data from a client request. It defines the default constructor. In addition, a method is provided to read bytes from the stream

The `ServletOutputStream` class extends `OutputStream`. It is implemented by the servlet container and provides an output stream that a servlet developer can use to write data to a client response. A default constructor is defined. It also

defines the `print()` and `println()` methods, which output data to the stream.

16. List classes that are provided in the `javax.servlet.http` package

Class	Description
<code>Cookie</code>	Allows state information to be stored on a client machine.
<code>HttpServlet</code>	Provides methods to handle HTTP requests and responses.
<code>HttpSessionEvent</code>	Encapsulates a session-changed event.
<code>HttpSessionBindingEvent</code>	Indicates when a listener is bound too run bound from a session value, or that a session attribute changed.

17. Write the usage of any two methods described in the `HttpServletResponse` interface

Method	Description
void <code>addCookie(Cookie cookie)</code>	Adds cookie to the HTTP response.
boolean <code>containsHeader(String field)</code>	Returns true if the HTTP response header contains a field named field.
String <code>encodeURL(String url)</code>	Determines if the session ID must be encoded in the URL identified as <code>url</code> . If so, returns the modified version of <code>url</code> . Otherwise, returns <code>url</code> . All URLs generated by a servlet should be processed by this method.
void <code>sendError(int c)</code> throws <code>IOException</code>	Sends the error code <code>c</code> to the client.
void <code>sendRedirect(String url)</code> throws <code>IOException</code>	Redirects the client to <code>url</code> .

void setDateHeader(String field, long msec)	Adds field to the header with date value equal to msec (milliseconds since midnight, January 1, 1970, GMT).
void setHeader(String field, String value)	Adds field to the header with value equal to value.
void setIntHeader(String field, int value)	Adds field to the header with value equal to value.
void setStatus(int code)	Sets the status code for this response to code.

18. What is Cookie? How it is helpful?

The **Cookie** class encapsulates a cookie. A *cookie* is stored on a client and contains state information. Cookies are valuable for tracking user activities.

For example, assume that a user visits an online store. A cookie can save the user's name, address, and other information. The user does not need to enter this data each time he or she visits the store.

19. Describe the significance of the valueBound and valueUnbound methods in the HttpSessionBindingListener interface

The **HttpSessionBindingListener** interface is implemented by objects that need to be notified when they are bound to or unbound from an HTTP session. The methods that are invoked when an object is bound or unbound are
void valueBound(HttpSessionBindingEvent e)
void valueUnbound(HttpSessionBindingEvent e) Here, e is the event object that describes the binding

20. What information can be stored in a cookie?

The names and values of cookies are stored on the user's machine. Some of the information that is saved for each cookie includes the following:

- The name of the cookie
- The value of the cookie
- The expiration date of the cookie
- The domain and path of the cookie

21. What is the significance of the getSession() method of HttpServletRequest

HttpSessionEvent encapsulates **session** events. It extends **EventObject** and is generated when a change occurs to the session.

It defines this constructor:

HttpSessionEvent(HttpSession session) Here, session is the source of the event.

HttpSessionEvent defines one method, **getSession()**, which is shown here: **HttpSession getSession()** It returns the session in which the event occurred.

22. Write the purpose of next() and getString().

boolean next() Throws SQLException	Moves the cursor forward one row from its current position.
String getString (int col_index) throws SQLException	Retrieves the value of the designated column in the current row of this ResultSet object as a String
String getString (String col_Label) throws SQLException	Retrieves the value of the designated column in the current row of this ResultSet object as a String in the Java programming language.

- The first time that the **next()** method of the **ResultSet** is called, the **ResultSet** pointer is positioned at the first row in the **ResultSet** and returns a boolean value that if false indicates that no rows are present in the **ResultSet**.
- A true value returned by the **next()** method means at least one row of data is present in the **ResultSet**, which causes the code to enter the do...while loop.
- The **getString()** method of the **ResultSet** object is used to copy the value of a specified column in the current row of the **ResultSet** to a **String** object.
- The **getString()** method is passed the name of the column in the **ResultSet** whose content needs to be copied, and the **getString()** method returns the value from the specified column.

You could also pass the number of the column to the **getString()** method instead of passing the column number.

23. List the parts of URL that is used in `getConnection()` method to establish connection.

<pre> public static Connection getConnection(String url,String user, String password)throws SQLException ex: String url = " jdbc:odbc:Customerinformation"; String userID = "jim "; String password = "keogh "; Statement DataRequest; private Connection Db; try { Class.forName("sun.jdbc.odbc.JdbcOdbcDriver"); Db= DriverManager. getConnection(url,userID,password); </pre>	Attempts to establish a connection to the given database URL. url - a database url of the form <code>jdbc:subprotocol:subname</code> user - the database user on whose behalf the connection is being made password - the user's password
---	---

24. Write the purpose of `setLoginTimeout()` and `getLoginTimeout()`

- `public static void DriverManager.setLoginTimeout(int seconds)` method can be used by the J2EE component to establish the maximum time the DriverManager waits for a response from a DBMS before timing out.
- This method is used to retrieve from the DriverManager the maximum time the DriverManager is set to wait until it times out. The `DriverManager.getLoginTimeout()` method returns an int that represents seconds

25. Write the purpose of `forName()` and `createStatement()`.

- The driver is loaded by calling the `Class.forName()` method and passing it the name of the driver, as shown in the following code segment
`Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");`
- Once a connection is obtained, we can interact with the database.
- The JDBC *Statement* interfaces define the methods and properties that enable you to send SQL or PL/SQL commands and receive data from your database.

`Statement st=con.createStatement();`

Interface is used for general-purpose access to your database, when using static SQL statements at runtime.

Statement `createStatement()`
throws *SQLException*
Creates a Statement object for sending SQL statements to the database.

26. What is the main purpose of the JDBC to ODBC (Type 1) driver?

- Type I driver provides mapping between JDBC and access API of a database
 - The access API calls the native API of the database to establish communication
- A common Type I driver defines a JDBC to ODBC bridge
 - ODBC is the database connectivity for databases
 - JDBC driver translates JDBC calls to corresponding ODBC calls
 - Thus if ODBC driver exists for a database this bridge can be used to communicate with the database from a Java application

28. Differentiate between Type 3 and Type 4 JDBC drivers. Which one is considered to be the fastest way to communicate SQL queries to the DBMS?

Type 3 JDBC drivers	Type 4 JDBC drivers
Type III driver make calls to a middleware component running on another server	Type IV driver is an all-Java driver that is also called a thin driver
This communication uses a database independent net protocol	It issues requests directly to the database using its native protocol
Middleware server then makes calls to the database using database-specific protocol	It can be used directly on platform with a JVM
The program sends JDBC call through the JDBC driver to the middle tier	Most efficient since requests only go through one layer
Middle-tier may use Type I or II JDBC driver to communicate with the database	Simplest to deploy since no additional libraries or middle-ware

29. What is the difference between the `executeQuery()` and `executeUpdate()` methods of a Statement object in JDBC?

1. **executeQuery():** This method returns the `ResultSet` object that contains rows, columns and metadata that represent data requested by the query.

- Generally, this method is used to execute only the 'SELECT' query of SQL
- Method Signature: `ResultSet executeQuery(String query);`
- For example: `Results= DataRequest.executeQuery (Select * from Customer);`

2. **executeUpdate():** This method is used to execute the queries that contain INSERT, DELETE and UPDATE statements

- Method returns integer indicating the number of rows that were updated by the query
 - Signature: `int executeUpdate(String query);`
- For example:
`int rows = st.executeUpdate("DELETE FROM EMPLOYEES WHERE STATUS=0");`

30. Briefly explain what a `ResultSet` object represents in JDBC.

- SQL statements that read data from a database query, return the data in a resultset
- **ResultSet** object maintains a cursor (virtual pointer) that points to the current row in the resultset
- Initially, cursor points to before the first row
- `next()` method used to move the cursor to the one row next from the current position
 - Returns true if row contains data, false otherwise
 - `getXxx()` method used to copy data from the row to a collection, object, or variable
 - Columns appear in the `ResultSet` in the order in which column names appear in the SELECT statement in the query

31. What is the main advantage of using a `PreparedStatement` object over a `Statement` object in JDBC?

- **PreparedStatement** object can be used to precompile and execute SQL query
- Query is created similar to other queries
- However, a question mark is given on the place for the value that is inserted into the query after it is compiled
- It is the value that changes each time the query is executed
- This type of the query is passed as the parameter to the `prepareStatement()` method of the `Connection` object which then returns the `PreparedStatement` object
- For example
`query= "select * from nation where population > ?"`
`PreparedStatement ps = prepareStatement(query);`
- Once the `PreparedStatement` object is obtained, the `setXxx()` methods is used to replace question mark with the value passed to `setXxx()` method

32. How do `PreparedStatement` objects handle dynamic values in SQL queries?

A `PreparedStatement` in Java allows dynamic values to be inserted into SQL queries safely and efficiently. Here's how it handles dynamic values:

- For example
`Query= "select * from nation where population > ?"`
`PreparedStatement ps = prepareStatement(query);`
- Once the `PreparedStatement` object is obtained, the `setXxx()` methods is used to replace question mark with the value passed to `setXxx()` method
- Each `setXxx()` method specifies the data type of value that is being passed. For example `ps.setInt(1, 100000);`
 - First parameter (int--identifies position of the question markplaceholder)
 - Second Parameter (value that replaces the question mark)
- Next, use appropriate execute method depending upon type of the query without any parameters
`ResultSet rs = ps.executeQuery();`

33. What are the different parameters used by `CallableStatement` object?

Three types of parameters exist: IN, OUT, and INOUT. The PreparedStatement object only uses the IN parameter. The CallableStatement object can use all the three.

Parameter	Description
IN	A parameter whose value is unknown when the SQL statement is created. You bind values to IN parameters with the setXXX() methods.
OUT	A parameter whose value is supplied by the SQL statement it returns. You retrieve values from the OUT parameters with the getXXX() methods.
INOUT	A parameter that provides both input and output values. You bind variables with the setXXX() methods and retrieve values with the getXXX() methods.

34. How to Insert a Row in to the ResultSet

- If ResultSet is updatable it is also possible to insert rows into it **result.moveToInsertRow();**
result.updateString (1, "Alex");
result.updateInt (2, 55);
result.insertRow(); **result.beforeFirst();**
 - The row pointed to after calling **moveToInsertRow()** is a special row, a buffer, which can be used to build up the row until all column values has been set on the row.
 - Once the row is ready to be inserted into the ResultSet, the **insertRow()** method is called
- Important to move the ResultSet to a valid position after inserting the new row

35. Why Savepoints are used in database transaction?

- The J2EE component can control the number of tasks that are rolled back by using savepoints.
- A savepoint is a virtual marker that defines the task at which the rollback stops.

36. How to batch sql statement into transaction statement?

- Another way to combine SQL statements into a transaction is to batch together these statements into a single transaction and then execute the entire transaction.
- You can do this by using the **addBatch()** method of the statement object.
- The **addBatch()** method receives a SQL statement as a parameter and places the SQL statement in the batch

37. What are the methods supported by RowSetListener class?

- rowSetChanged(RowSetEvent event):** This method is called when the data in the RowSet object has changed, typically because the RowSet has been re-executed or its data has been modified while connected to the database.
- rowChanged(RowSetEvent event):** This method is called when a row in the RowSet object has been changed, indicating that one or more column values in the row have been modified.
- cursorMoved(RowSetEvent event):** This method is called when the cursor position within the RowSet object has been moved, indicating that the current row has changed.

38. What is JavaServerPages?

Java Server Pages (JSP) is a technology for developing Web pages that supports dynamic content. This helps developers insert java code in HTML pages by making use of special JSP tags, most of which start with <% and ends with %>

39. List any four advantages of using JSP

JSP vs. Active Server Pages(ASP)

- The advantages of JSP are twofold.
- First, the dynamic part is written in Java, not Visual Basic or other MS specific language, so it is more powerful and easier to use.
- Second, it is portable to other operating systems and non-Microsoft Web servers.

JSP vs. Pure Servlets

It is more convenient to write (and to modify!) regular HTML than to have plenty of println statements that generate the HTML.

JSP vs. Server-Side Includes (SSI)

SSI is really only intended for simple inclusions, not for "real" programs that use form data, make database connections, and the like.

JSP vs. JavaScript

JavaScript can generate HTML dynamically on the client but can hardly interact with the web server to perform complex tasks like database access and image processing etc.

JSP vs. Static HTML Regular HTML cannot contain dynamic information

40. List any four implicit Objects

- **request**- This is the **HttpServletRequest** object associated with the request.
- **response**- This is the **HttpServletResponse** object associated with the response to the client.
- **out**- This is the **PrintWriter** object used to send output to the client.
- **session**- This is the **HttpSession** object associated with the request.
- **application**- This is the **ServletContext** object associated with application context.
- **config**- This is the **ServletConfig** object associated with the page.
- **pageContext**- This encapsulates use of server-specific features like higher performance **JspWriters**.
- **Exception**- The Exception object allows the exception data to be accessed by designated JSP

41. What is the usage of buffer attribute in JSP?

- The **buffer attribute** specifies the buffering characteristics for the server output response object.
- You may code a value of "none" to specify no buffering so that the servlet output is immediately directed to the response object or you may code a maximum buffer size in kilobytes, which directs the servlet to write to the buffer before writing to the response object.
- To direct the servlet to write the output directly to the response output object, use the following:

```
<%@ page buffer="none" %>
```

Use the following to direct the servlet to write the output to a buffer of size not less than 8 kilobytes:

```
<%@ page buffer="8kb" %>
```

42. What is page Directive?

The page directive is used to provide instructions to the container. These instructions pertain to the current JSP page. You may code page directives anywhere in your JSP page. By convention, page directives are coded at the top of the JSP page. Following is the basic syntax of the page directive: `<%@ page attribute="value" %>`

You can write the XML equivalent of the above syntax as follows: `<jsp:directive.page attribute="value"/>`

(4 to 6 marks)

1. Explain the three key methods in the lifecycle of a servlet (init(), service(), destroy())

- Three methods are central to the life cycle of a servlet.
- These are `init()`, `service()`, and `destroy()`.
- They are implemented by every servlet and are invoked at specific times by the server.
- Let us consider a typical user scenario to understand when these methods are called.
- **First**, assume that a user enters a Uniform Resource Locator (URL) to a web browser. The browser then generates an HTTP request for this URL. This request is then sent to the appropriate server
- **Second**, this HTTP request is received by the web server. The server maps this request to a particular servlet. The servlet is dynamically retrieved and loaded into the address space of the server
- **Third**, the server invokes the `init()` method of the servlet. This method is invoked only when the servlet is first loaded into memory. It is possible to pass initialization parameters to the servlet so it may configure itself.
- **Fourth**, the server invokes the `service()` method of the servlet. This method is called to process the HTTP request. You will see that it is possible for the servlet to read data that has been provided in the HTTP request. It may also formulate an HTTP response for the client

- The servlet remains in the server's address space and is available to process any other HTTP requests received from clients. The **service()** method is called for each HTTP request
- Finally, the server may decide to unload the servlet from its memory.
- The algorithms by which this determination is made are specific to each server.
- The server calls the **destroy()** method to relinquish any resources such as file handles that are allocated for the servlet.
- Important data may be saved to a persistent store.

2. Develop a basic servlet program that displays a welcome message to the user

HelloServlet.java

```
import java.io.*;
import javax.servlet.*;
public class HelloServlet extends GenericServlet
{
    public void service(ServletRequest request, ServletResponse
    response) throws ServletException, IOException
    {
        response.setContentType("text/html");
        PrintWriter pw = response.getWriter();
        pw.println("Hello world welcome!"); pw.close();
    }
}
web.xml
<servlet>
<servlet-name>HelloServlet</servlet-name>
<servlet-class>HelloServlet</servlet-class>
</servlet>
<servlet-mapping>
<servlet-name>HelloServlet</servlet-name>
<url-pattern>/a</url-pattern>
</servlet-mapping>
```

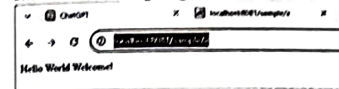
Compile the HelloServlet.java file

C:\Program Files (x86)\Apache Software Foundation\Tomcat 8.5\webapps\sample\WEB-INF\classes>javac HelloServlet.java -cp

"C:\Program Files (x86)\Apache Software Foundation\Tomcat 8.5\lib\servlet-api.jar"

It creates the HelloServlet.class file in the directory. Run the tomcat server and type the following in the browser

Now run the program in the browser



3. Explain the functionalities of the following methods available in servletConfig Interface The ServletConfig Interface

- The ServletConfig interface allows a servlet to obtain configuration data when it is loaded. The methods declared by this interface are summarized here:

Method	Description
Servlet Context getServletContext()	Returns the context for this servlet.
String getInitParameter(Stringparam)	Returns the value of the initialization parameter named param.
Enumeration getInitParameterNames()	Returns an enumeration of all initialization parameter names.
String getServletName()	Returns the name of the invoking servlet.

4. Write the usage of any four methods of servletRequest interface

• The ServletRequest Interface

- The ServletRequest interface enables a servlet to obtain information about a client request. Several of its methods are shown in Table

Method	Description
Object getAttribute(String attr)	Returns the value of the server attribute named attr.
String getMimeType(String)	Returns the MIME type of file.

file) String getRealPath(String vpath)	Returns the real path that corresponds to the virtual path vpath.
String getServerInfo()	Returns information about the server.
void log(String s)	Writes s to the servlet log.
void log(String s, Throwable e)	Writes s and the stack trace for e to the servlet log.
void setAttribute(String attr, Object val)	Sets the attribute specified by attr to the value passed in val.

5. Write the usage of any four methods of servletResponse interface

The ServletResponse Interface

- The ServletResponse interface enables a servlet to formulate a response for a client. Several of its methods are shown in Table

Method	Description
Object getAttribute(String attr)	Returns the value of the attribute named attr.
String getCharacterEncoding()	Returns the character encoding of the request.
int getContentLength()	Returns the size of the request. The value -1 is returned if the size is unavailable.
String getContentType()	Returns the type of the request. A null value is returned if the type cannot be determined.
ServletInputStream getInputStream() throws IOException	Returns a ServletInputStream that can be used to read binary data from the request. An IllegalStateException is thrown if getReader() has already been invoked for this request.
String getParameter(String pname)	Returns the value of the parameter named pname.
Enumeration getParameterNames()	Returns an enumeration of the parameter names for this request.

6. Write a Java servlet named `FormProcessorServlet` that can handle HTTP POST requests containing form data. The servlet should: **Read Parameters:** Extract the names and values of parameters submitted from the form. **Process Data:** Access the specific parameter values for fields like "name" and "email" (assuming those are the form field names). **Generate Response:** Create an HTML response that displays the submitted data back to the user in a user-friendly format.

Index.html

```
<html>      <head><title>Form</title>      </head>
<body> <center>
<form      name="Form1"      method="post"
action="http://localhost:8081/sample/a">
  <table> <tr><td><b>Employee</b></td>
<td><input type="text" name="e" size="25" value=""></td>
</tr> <tr> <td><b>Email</b></td>
<td><input type="text" name="p" size="50" value=""></td>
</tr> </table>
<input type="submit" value="Submit">
</form> </center> </body> </html>
```

Web.xml

```
<web-app>
<servlet>
<servlet-name>FormProcessorServlet</servlet-name>
<servlet-class>FormProcessorServlet</servlet-class>
</servlet>
<servlet-mapping>
<servlet-name>FormProcessorServlet</servlet-name>
<url-pattern>/a</url-pattern>
</servlet-mapping>
</web-app>
```

formProcessorServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class FormProcessorServlet extends HttpServlet {
  public void doPost(HttpServletRequest request,
    HttpServletResponse response) throws ServletException,
    IOException {
    String name = request.getParameter("e");
```

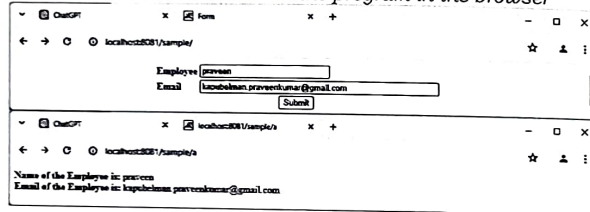


```
String email = request.getParameter("p");
response.setContentType("text/html");
PrintWriter pw = response.getWriter();
pw.println("<html><body>");
pw.println("<b>Name of the Employee is: </b>" + name +
"<br>");
pw.println("<b>Email of the Employee is: </b>" + email);
pw.println("</body></html>");
pw.close();
}
```

Compile the program:

```
C:\Program Files (x86)\Apache Software Foundation\Tomcat
8.5\webapps\sample\WEB-INF\classes>javac
FormProcessorServlet.java -cp "C:\Program Files
(x86)\Apache Software Foundation\Tomcat 8.5\lib\servlet-
api.jar"
```

Start the tomcat server and run the program in the browser



7. Explain the following methods of HttpServletRequest interface with their syntax

- i. `getCookies`
- j. `getMethod`
- k. `getPathInfo`
- l. `getSession`

Method	Description
<code>Cookie[]getCookies()</code>	Returns an array of the cookies in this request.
<code>String getMethod()</code>	Returns the HTTP method for this request.
<code>String getPathInfo()</code>	Returns any path information that is located after the servlet path and before a query string of

	theURL.
<code>HttpSession getSession()</code>	Returns the session for this request. If a session does not exist, one is created and then returned.
<code>HttpSession getSession(boolean new)</code>	If new is true and no session exists, creates and returns a session for this request. Otherwise, returns the existing session for this request.

8. With an example, Explain the purpose and behavior of the doGet() method in a servlet class.

- The `doGet( )` method is overridden to process any HTTP GET requests that are sent to this servlet.
- It uses the `getParameter( )` method of `HttpServletRequest` to obtain the selection that was made by the user. A response is then formulated. The example contains two files. A webpage is defined in `index.html`, and a servlet is defined in `ColorGetServlet.java`

index.html

```
<html><body><center>
<formname="Form.1" action="http://localhost:8080/servlets-
examples/servlet/ColorGetServlet">
<B>Color:</B>
<selectname="color"size="1">
<optionvalue="Red">Red</option>
<optionvalue="Green">Green</option>
<optionvalue="Blue">Blue</option>
</select><br><br><inputtype=submitvalue='Submit'>
</form></body></html>
```

ColorGetServlet.java

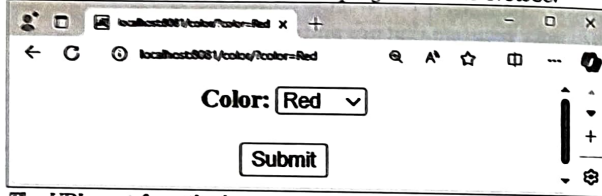
```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class ColorGetServlet extends HttpServlet {
public void doGet(HttpServletRequest request,
HttpServletResponse response) throws
ServletException,IOException{
```

```
String color=request.getParameter("color");
response.setContentType("text/html");
PrintWriter pw=response.getWriter();
pw.println("<B> The selected color is: ");
pw.println(color);
pw.close();
```

Compile the program

```
C:\Program Files (x86)\Apache Software Foundation\Tomcat
8.5\webapps\color\WEB-INF\classes>javac ColorGetServlet.java
-cp "C:\Program Files (x86)\Apache Software Foundation\Tomcat
8.5\lib\servlet-api.jar"
```

Start the tomcat server and run the program in the browser



The URL sent from the browser to the server is

The characters to the right of the question mark are known as the query string.

9. With an example, Explain the purpose and behavior of the doPost() method in a servlet class.

The doPost() method is overridden to process any HTTP POST requests that are sent to this servlet.

It uses the getParameter() method of HttpServletRequest to obtain the selection that was made by the user. A response is then formulated

index.html

```
<html><head> <title>Color Form</title></head>
<body> <center>
<form name="Form1" method="post"
action="http://localhost:8081/color/color">
<h2>Select a Color:</h2>
<select name="color">
<option value="Red">Red</option>
```

```
<option value="Green">Green</option>
<option value="Blue">Blue</option>
</select> <br><br>
<input type="submit" value="Submit">
</form> </center></body></html>
```

ColorPostServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

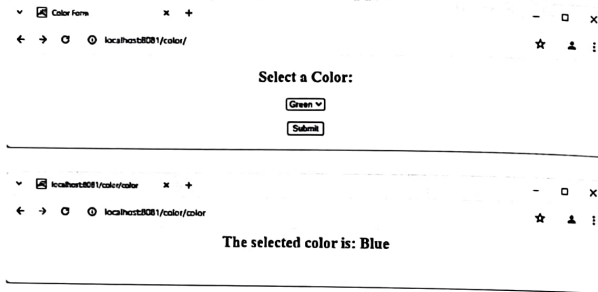
public class ColorPostServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response) throws ServletException,
        IOException {
        // Get the selected color from the request parameters
        String color = request.getParameter("color");
        System.out.println("Selected color: " + color);
        // Debugging statement
        // Set the response content type
        response.setContentType("text/html");
        // Get PrintWriter object
        PrintWriter out = response.getWriter();
        // Write HTML response
        out.println("<html><body>");
        out.println("<center><h2>The selected color is: " + color +
            "</h2></center>");
        out.println("</body></html>");
        // Close PrintWriter
        out.close();
    }
}
```

Web.xml

```
<web-app>
<servlet>
    <servlet-name>ColorPostServlet</servlet-name>
    <servlet-class>ColorPostServlet</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>ColorPostServlet</servlet-name>
    <url-pattern>/color/</url-pattern>
</servlet-mapping>
</web-app>
```

Compile the file C:\Program Files (x86)\Apache Software Foundation\Tomcat 8.5\webapps\color\WEB-INF\classes>javac ColorPostServlet.java -cp "C:\Program Files (x86)\Apache Software Foundation\Tomcat 8.5\lib\servlet-api.jar"

Start the Tomcat server and run the application in browser



10. What are the different methods involved in the process of session management in servlets? Explain their purpose

- A session can be created via the **getSession()** method of **HttpServletRequest**. An **HttpSession** object is returned. This object can store a set of bindings that associate names with objects.
- The **setAttribute()**, **getAttribute()**, **getAttributeNames()**, and **removeAttribute()** methods of **HttpSession** manage these bindings.
- A new session is created if one does not already exist.
- The **getAttribute()** method is called to Obtain the object that is bound to the name "date".
- That object is a **Date** object that encapsulates the date and time when this page was last accessed
- A **Date** object encapsulating the current date and time is then created.
- The **setAttribute()** method is called to bind the name "date" to this object.

```
import java.io.*;
import java.util.*;
```

```
import javax.servlet.*;
import javax.servlet.http.*;

public class DateServlet extends HttpServlet {
    public void doGet(HttpServletRequest request,
        HttpServletResponse response) throws ServletException,
        IOException {
        // Get the HttpSession object.
        HttpSession hs = request.getSession(true);
        // Get writer.
        response.setContentType("text/html");
        PrintWriter pw = response.getWriter();
        pw.print("<B>");
        // Display date/time of last access.
        Date date = (Date)hs.getAttribute("date"); if(date != null) {
        pw.print("Last access: " + date + "<br>");
        } // Display current date/time. date = new Date();
        hs.setAttribute("date", date);
        pw.println("Current date: " + date);
    }
}
```

When you first request this servlet, the browser displays one line with the current date and time information. On subsequent invocations, two lines are displayed. The first line shows the date and time when the servlet was last accessed. The second line shows the current date and time.

11. How cookies can be handled using servlet

- When this button is pressed, the value in the textfield is sent to **AddCookieServlet** via an HTTP POST request.

AddCookie.html

```
<html><body><center><form name="Form1" method="post"
action="http://localhost:8080/servletexamples/servlet/Add
CookieServlet">
<B>Enter a value for MyCookie:</B>
<input type="text" name="data" size=25 value="">
<input type="submit" value="Submit">
</form></body></html>
```

AddCookieServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class AddCookieServlet extends HttpServlet {
```



```

public void doPost(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {
    // Get parameter from HTTP request.
    String data = request.getParameter("data");
    // Create cookie.
    Cookie cookie = new Cookie("MyCookie", data);
    // Add cookie to HTTP response.
    response.addCookie(cookie);
    // Write output to browser.
    response.setContentType("text/html");
    PrintWriter pw = response.getWriter();
    pw.println("<B>MyCookie has been set to");
    pw.close();
}
}

```

- It gets the value of the parameter named "data". It then creates a **Cookie** object that has the name "MyCookie" and contains the value of the "data" parameter.
- The cookie is then added to the header of the HTTP response via the **addCookie()** method. A feedback message is then written to the browser.

GetCookiesServlet.java

```

import java.io.*; import javax.servlet.*;
import javax.servlet.http.*;
public class GetCookiesServlet extends HttpServlet {
    public void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {
        // Get cookies from header of HTTP request.
        Cookie[] cookies = request.getCookies();
        // Display these cookies.
        response.setContentType("text/html");
        PrintWriter pw = response.getWriter();
        pw.println("<B>");
        for(int i = 0; i < cookies.length; i++) {
            String name = cookies[i].getName();
            String value = cookies[i].getValue();
            pw.println("name = " + name + "; value = " + value);
        } pw.close(); } }

```

- It invokes the **getCookies()** method to read any cookies that are included in the HTTP GET request. The names and values of these cookies are then written to the HTTP response.
- Observe that the **getName()** and **getValue()** methods are called to obtain this information.

12. Explain why JSP is a compelling choice for web development compared to the Common Gateway Interface (CGI).

Java Server Pages often serve the same purpose as programs implemented using the Common Gateway Interface (CGI). But JSP offers several advantages in comparison with the CGI.

- Performance is significantly better because JSP allows embedding Dynamic Elements in HTML Pages itself instead of having separate CGI files.
- JSP are always compiled before they are processed by the server unlike CGI/Perl which requires the server to load an interpreter and the target script each time the page is requested.
- Java Server Pages are built on top of the Java Servlets API, so like Servlets, JSP also has access to all the powerful Enterprise Java APIs, including JDBC, JNDI, EJB, JAXP, etc.
- JSP pages can be used in combination with servlets that handle the business logic, the model supported by Java servlet template engines.
- Finally, JSP is an integral part of Java EE, a complete platform for enterprise class applications. This means that JSP can play a part in the simplest applications to the most complex and demanding

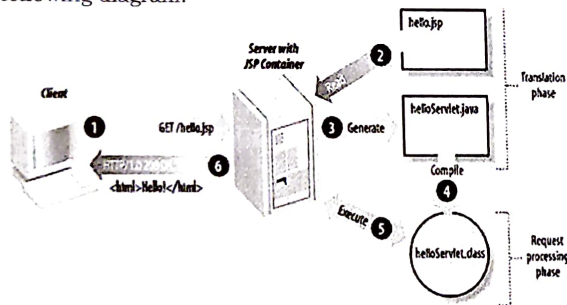
13. Explain the key steps involved in how a web server processes a JSP request and generates a web page.

JSP Processing: The following steps explain how the web server creates the Web page using JSP:

- As with a normal page, browser (client) sends an HTTP request to the web server.
- The web server recognizes that the HTTP request is for a JSP page and forwards it to a JSP engine. This is done by

using the URL or JSP page which ends with **.jsp** instead of **.html**.

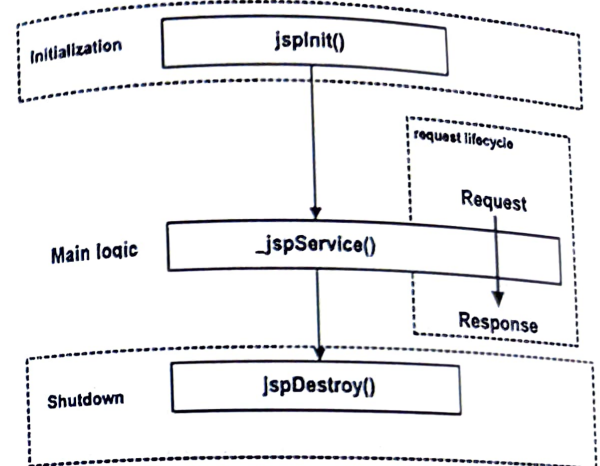
- The JSP engine loads the JSP page from disk and converts it into a servlet content. This conversion is very simple in which all template text is converted to **println()** statements and all JSP elements are converted to Java code. This code implements the corresponding dynamic behavior of the page.
- The JSP engine compiles the servlet into an executable class and forwards the original request to a servlet engine.
- Apart of the web server called the servlet engine loads the Servlet class and executes it. During execution, the servlet produces an output in HTML format. This output is further passed on to the web server by the servlet engine inside an HTTP response.
- The web server forwards the HTTP response to our browser in terms of static HTML content.
- Finally, the web browser handles the dynamically-generated HTML page inside the HTTP response exactly as if it were a static page. All the above mentioned steps can be seen in the following diagram:



14. Explain the key stages involved in the lifecycle of a Java Server Page (JSP).

Key Stages involved in the Lifecycle of a Java Server Page
The following are the paths followed by a JSP

- Compilation
- Initialization
- Execution
- Cleanup



JSP Compilation: When a browser asks for a JSP, the JSP engine first checks to see whether it needs to compile the page. If the page has never been compiled, or if the JSP has been modified since it was last compiled, the JSP engine compiles the page. The compilation process involves three steps:

- Parsing the JSP.
- Turning the JSP in to a servlet.
- Compiling the servlet.

JSP Initialization: When a container loads a JSP it invokes the `jspInit()` method before servicing any requests. If we need to perform JSP-specific initialization, override the `jspInit()` method:

```
public void jspInit()  
{ //Initializationcode...  
}
```

Typically, initialization is performed only once and as with the servlet init method, we generally initialize database connections, open files, and create lookup tables in the `jspInit` method.

JSP Execution: This phase of the JSP life cycle represents all interactions with requests until the JSP is destroyed.

- Whenever a browser requests a JSP and the page has been loaded and initialized, the JSP engine invokes the `_jspService()` method in the JSP.

- The `_jspService()` method takes an `HttpServletRequest` and an `HttpServletResponse` as its parameters as follows:

```
void    _jspService(HttpServletRequest request,
        HttpServletResponse response)
{
    //Servicehandlingcode...
}
```

- The `_jspService()` method of a JSP is invoked on request basis.
- This is responsible for generating the response for that request and this method is also responsible for generating responses to all seven of the HTTP methods, i.e., **GET, POST, DELETE, etc**

JSP Cleanup: The destruction phase of the JSP lifecycle represents when a JSP is being removed from use by a container.

- The `jspDestroy()` method is the JSP equivalent of the destroy method for servlets.
- Override `jspDestroy` when you need to perform any **cleanup**, such as releasing database connections or closing open files. The `jspDestroy()` method has the following form:

```
public void jspDestroy()
{
    //Yourcleanupcodegoeshere.
}
```

15. Write short note on JSP directives and JSP Actions

JSP Directives: A JSP directive affects the overall structure of the servlet class. It usually has the following form: `<%@directive attribute="value"%>`

There are three types of directive tag:

Directive	Description
<code><%@ page... %></code>	Defines page-dependent attributes, such as scripting language, error page and buffering requirements.
<code><%@ include... %></code>	Includes a file during the translation phase.
<code><%@ taglib... %></code>	Declares a tag library, containing custom actions, used in the page

16. Explain any four buffer attributes with examples

The buffer Attribute: The buffer attribute specifies the buffering characteristics for the server output response object. You may code a value of **"none"** to specify no buffering so that the servlet output is immediately directed to the response object or you may code a maximum buffer size in kilobytes, which directs the servlet to write to the buffer before writing to the response object.

To direct the servlet to write the output directly to the response output object, use the following:

```
<%@ page buffer="none" %>
```

Use the following to direct the servlet to write the output to a buffer of size not less than 8 kilobytes:

```
<%@ page buffer="8kb" %>
```

17. Explain the steps in the JDBC process.

steps in the JDBC process includes:

- Loading the JDBC driver
- Connecting to the DBMS
- Creating and executing a statement
- Processing data returned by the DBMS
- Terminating the connection with the DBMS

Step 1: Loading the JDBC Driver

- The JDBC driver must be loaded before the J2EE component can connect to the DBMS.
- The `Class.forName()` method is used to load the JDBC driver.
- The developer must write a routine that loads the JDBC / ODBC Bridge driver called `sun.jdbc.odbc.JdbcOdbcDriver`.
- The driver is loaded by calling the `Class.forName()` method and passing it the name of the driver, as shown in the following code segment

```
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
```

Step 2: Connect to the DBMS

- Once the driver is loaded, the J2EE component must connect to the DBMS using the `DriverManager.getConnection()` method.
- The `java.sql.DriverManager` class is the highest class in the `java.sql` hierarchy and is responsible for managing driver information.
- Method: DriverManager*

public static Connection getConnection(String url) throws SQLException	Attempts to establish a connection to the given database URL. The DriverManager attempts to select an appropriate driver from the set of registered JDBC drivers.
public static Connection getConnection(String url,String user, String password)throws SQLException	Attempts to establish a connection to the given database URL. url - a database url of the form jdbc:subprotocol:subname user - the database user on whose behalf the connection is being made password - the user's password

```
String url = "jdbc:odbc:Customerinformation";
String userID = "jim ";
String password = "keogh ";
Statement DataRequest ;
private Connection Db ;
try {
Class.forName( "sun.jdbc.odbc.JdbcOdbcDriver" );
Db= DriverManager . getConnection(url,userID,password) ;
}
```

Step 3: Creating statement

- Once a connection is obtained, we can interact with the database.
- The JDBC *Statement* interfaces define the methods and properties that enable you to send SQL or PL/SQL commands and receive data from your database.

```
Statement st=con.createStatement();
```

Step 3: Executing Statement

Once you've created a Statement object, you can then use it to execute an SQL statement with its execute methods

Syntax: ResultSet rs=st.executeQuery("query");

Example:

```
ResultSet rs = stmt.executeQuery("SELECT * from Customer");
```

Step 4: Processing data returned by the DBMS

The **java.sql.ResultSet** object consists of methods used to interact with data that is returned by the DBMS to the J2EE component. **Method: ResultSet**

boolean next() Throws SQLException	Moves the cursor forward one row from its current position.
String getString (int col_index) throws SQLException	Retrieves the value of the designated column in the current row of this ResultSet object as a String
String getString (String col_Label) throws SQLException	Retrieves the value of the designated column in the current row of this ResultSet object as a String in the Java programming language.
int getInt (int columnIndex) throws SQLException	Returns the int in the current row in the specified column index.
int getInt (String columnLabel) throws SQLException	Retrieves the value of the designated column in the current row

- Example:

```
while(rs.next()){
System.out.println(rs.getString(1));
System.out.println(rs.getInt("emp_id"));
}
```

Step 5: Terminating Connection with DBMS

- The connection of DBMS is terminated by using close() method.
- rs.close();** Releases this ResultSet object's database and JDBC resources immediately
- st.close();** Releases this Statement object's database and JDBC resources immediately
- con.close();** Releases this Connection object's database and JDBC resources immediately

18. Write the steps to associate database with JDBC/ODBC bridge.

Associating the JDBC/ODBC Bridge with the Database:
You use the ODBC Data Source Administrator to create the association between the database and the JDBC/ ODBC bridge. Here's what you need to do:

1. Select Start | Settings, and then the Control Panel.
2. Select ODBC 32 to display the ODBC Data Source Administrator.
3. Add a new user by selecting the Add button.
4. Select the driver then select Finish. Use the Microsoft Access Driver if you are using Microsoft Access; otherwise, select the driver for the DBMS that you are using. If you don't find the driver for your DBMS on the list, you'll need to install the driver. Contact the manufacturer of the DBMS for more information on how to obtain the driver.
5. Enter the name of the database as the Data Source name in the ODBC Microsoft Access Setup dialog box. This is the name that will be used within your Java database program to connect to the DBMS.
6. Enter a description for the data source. This is optional, but will be a reminder of the kind of data that is stored in the database.
7. Click the Select button: You'll be prompted to browse the directory of each hard drive connected to your computer in order to define the direct path to the database. Click OK once you locate the database, and the directory path and the name of the database will be displayed in the ODBC Microsoft Access Setup dialog box.
8. Since this is your database, you can determine if a login name and password is required to access the database.
9. If so, then click the Advanced button to display the Set Advanced Options dialog box. This dialog box is used to assign a login name, also referred to as a user ID, and a password to the database. Select OK. If not, then skip this step.
10. When the ODBC Microsoft Access Setup dialog box appears, select OK.
11. Select OK to close the ODBC Data Source Administrator dialog box

19. What is JDBC Driver? Explain different types.

JDBC Drivers: JDBC uses drivers to translate generalized JDBC calls into vendor-specific database calls. Drivers exist for most popular databases. Four Classes of JDBC drivers exist.

Type I JDBC-to-ODBC Driver

Type II Java/Native Code Driver

Type III JDBC Driver

Type IV JDBC Driver

Drivers (Type I): Type I driver provides mapping between JDBC and access API of a database

- The access API calls the native API of the database to establish communication
- A common Type I driver defines a JDBC to ODBC bridge
 - ODBC is the database connectivity for databases
 - JDBC driver translates JDBC calls to corresponding ODBC calls
 - Thus if ODBC driver exists for a database this bridge can be used to communicate with the database from a Java application
- Inefficient and narrow solution
 - Inefficient, because it goes through multiple layers
 - Narrow, since functionality of JDBC code limited to whatever ODBC supports

Drivers (Type II): Type II driver communicates directly with native API

- Type II makes calls directly to the native API calls
- More efficient since there is one less layer to contend with (i.e. no ODBC)
- It is dependent on the existence of a native API for a database

Drivers (Type III): Type III driver makes calls to a middleware component running on another server

- This communication uses a database independent net protocol
- Middleware server then makes calls to the database using database-specific protocol
- The program sends JDBC call through the JDBC driver to the middle tier
- **Middle-tier may use Type I or II JDBC driver to communicate with the database.**

Drivers (Type IV): Type IV driver is an all-Java driver that is also called a thin driver

- It issues requests directly to the database using its native protocol
- It can be used directly on platform with a JVM

- Most efficient since requests only go through one layer
- Simplest to deploy since no additional libraries or middle-ware

20. Explain the steps to connect to the database in java?

- **Choose a Database:** First, you need to choose the database you want to connect to. Common options include MySQL, PostgreSQL, Oracle, SQLite, etc.
- **Download the JDBC Driver:** Each database has its own JDBC (Java Database Connectivity) driver that allows Java applications to interact with the database. Download the appropriate JDBC driver for your chosen database and include it in your project's classpath.
- **Import Necessary Packages:** Import the necessary JDBC packages in your Java program. These typically include `java.sql.*` and sometimes additional packages specific to the database you're using.
- **Load the Driver:** Use `Class.forName()` to load the JDBC driver class. This step is necessary to register the driver with the `DriverManager`.
- **Establish Connection:** Use the `DriverManager.getConnection()` method to establish a connection to the database. You need to provide the URL of the database, username, and password as parameters.
- **Create a Statement:** Once the connection is established, create a `Statement` or a `PreparedStatement` object. `Statements` are used to execute SQL queries.
- **Execute SQL Queries:** Use the `executeQuery()` method to execute `SELECT` queries or `executeUpdate()` method for `INSERT`, `UPDATE`, `DELETE`, and `DDL` (Data Definition Language) queries.
- **Process Results:** If you've executed a `SELECT` query, you can process the results returned by the database. Use methods like `ResultSet.next()` to iterate through the result set and retrieve data.
- **Close Resources:** After you're done with the database operations, make sure to close the `ResultSet`, `Statement`, and `Connection` objects to release the database and JDBC resources. Use `try-with-resources` or explicitly close them in a finally block to ensure proper resource management.

Example:

```
String url = "jdbc:odbc:Customerinformation";
Statement DataRequest;
Connection Db;
try{
    Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
    Db=DriverManager.getConnection(url);
} catch(ClassNotFoundException e){
    System.err.println("Unable to load the driver" + e);
    System.exit(1);
} catch(SQLException e){
    System.err.println("Cannot connect to the database"+ e);
    System.exit(1);
}
```

21. What are the JDBC statements? Explain

In JDBC (Java Database Connectivity), statements are objects used to execute SQL queries against a database. There are mainly three types of statements in JDBC:

1. Statement: This interface is used to execute simple SQL queries without parameters. It can be used for executing queries such as `SELECT`, `INSERT`, `UPDATE`, `DELETE`, and `DDL` (Data Definition Language) statements.

```
Statement statement = connection.createStatement();
ResultSet resultSet = statement.executeQuery("SELECT * FROM my_table");
```

2. PreparedStatement: This interface extends `Statement` and provides additional methods for executing parameterized SQL queries. `PreparedStatement` is precompiled, which can improve performance when executing the same SQL statement multiple times with different parameters.

```
PreparedStatement preparedStatement =
    connection.prepareStatement("INSERT INTO my_table (column1,
column2) VALUES (?, ?)");
preparedStatement.setString(1, "value1");
preparedStatement.setString(2, "value2");
int rowsAffected = preparedStatement.executeUpdate();
```

3. Callable Statement: This interface is used to execute stored procedures or functions in the database. `CallableStatement` allows you to call database stored procedures or functions with `IN`, `OUT`, and `INOUT` parameters.

```
CallableStatement callableStatement =
connection.prepareCall("{call my_stored_procedure(?, ?)}");
callableStatement.setInt(1, parameter1);
callableStatement.registerOutParameter(2, Types.INTEGER);
callableStatement.execute();
int returnValue = callableStatement.getInt(2);
```

Statement: Suitable for executing simple SQL queries without parameters.

PreparedStatement: Suitable for executing parameterized SQL queries, which are precompiled for better performance.

CallableStatement: Suitable for executing stored procedures or functions in the database, supporting IN, OUT, and INOUT parameters.

22. Describe the following

i) Scrollable ResultSet ii) Updatable ResultSet

Scrollable ResultSet: A Scrollable ResultSet in JDBC is a type of ResultSet that allows you to move the cursor forward and backward, as well as to move to a specific row or to a relative position from the current row. By default, ResultSets in JDBC are forward-only, meaning you can only traverse them in one direction - forward from the first row to the last. However, if you need more flexibility in navigating the result set, you can create a scrollable result set.

• Three ResultSet types:

• TYPE_FORWARD_ONLY (default type)

- ResultSet can only be navigated forward, move from row 1, to row 2, to row 3 etc

• TYPE_SCROLL_INSENSITIVE

- ResultSet can be navigated (scrolled) both forward and backwards
- Can also jump to a position relative to the current position, or jump to an absolute position
- The ResultSet is insensitive to changes in the underlying data source
- if a record in the ResultSet is changed in the database by another thread or process, it will not be reflected in already opened ResultSets of this type

• TYPE_SCROLL_SENSITIVE

- ResultSet can be navigated (scrolled) both forward and backwards
- can also jump to a position relative to the current position, or jump to an absolute position
- The ResultSet is sensitive to changes in the underlying data source
- if a record in the ResultSet is changed in the database by another thread or process, it will be reflected in already opened ResultSets of this type.

Navigation Methods

boolean first() throws SQLException	Moves the cursor to the first row.
boolean last() throws SQLException	Moves the cursor to the last row.
boolean next() throws SQL Exception	Moves the cursor to the next row. This method returns false if there are no more rows in the result set.
boolean previous() throws SQLException	Moves the cursor to the previous row. This method returns false if the previous row is off the result set.
boolean absolute(int row) throws SQLException	Moves the cursor to the specified row.
boolean relative(int row) throws SQLException	Moves the cursor the given number of rows forward or backward, from where it is currently pointing.
int getRow() throws SQLException	Returns the row number that the cursor is pointing to.

Example:

```
String query= "SELECT * FROM Customers";
```

```
DataRequest=Db.createStatement(TYPE_SCROLL_INSENSITIVE;
```

```
DataRequest.setFetchSize(500);
```

```
Results = DataRequest.executeQuery (query);
```

```
Results.first(); Results.last(); Results.previous() ;
```

```
Results.absolute (10); Results.relative(-2); Results. relative(2) ;
```


Updatable ResultSet: An updatable ResultSet is a feature provided by JDBC (Java Database Connectivity) that allows you to update the rows retrieved from a database table through a ResultSet object.

ResultSet concurrency determines whether the ResultSet can be updated, or only read

- Not all databases and JDBC drivers support that the ResultSet is updated
- ResultSet can have one of two concurrency levels
 - CONCUR_READ_ONLY - means the ResultSet can only be read
 - CONCUR_UPDATABLE - means that the ResultSet can be both read and updated

Here's a general outline of how you can create an updatable ResultSet in Java:

1. Create a Statement object with the appropriate parameters:

```
Statement statement =  
connection.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,  
ResultSet.CONCUR_UPDATABLE);  
Here, TYPE_SCROLL_SENSITIVE indicates that the ResultSet  
is scrollable and sensitive to changes made by others, and  
CONCUR_UPDATABLE indicates that the ResultSet is  
updatable.
```

2. Execute a query:

```
ResultSet resultSet = statement.executeQuery("SELECT * FROM  
your_table");
```

3. Update the ResultSet: You can use various methods provided by the ResultSet interface such as **updateString()**, **updateInt()**, etc., to modify the data in the ResultSet.

```
while (resultSet.next())  
{  
    resultSet.updateString("column_name", "new_value");  
    resultSet.updateRow();  
} // This updates the row in the database
```

23. Write a note on DatabaseMetaData interface Metadata

- Metadata is data about data.
- A J2EE component can access metadata by using the *DatabaseMetaData* interface.
- The *DatabaseMetaData* interface is used to retrieve information about databases, tables, columns, and indexes among other information about the DBMS.
- A J2EE component retrieves metadata about the database by calling the *getMetaData()* method of the *Connection* object.
- The *getMetaData()* method returns a *DatabaseMetaData* object that contains information about the database and its components.
- Once the *DatabaseMetaData* object is obtained, an assortment of methods contained in the *DatabaseMetaData* object are called to retrieve specific metadata.
- Here are some of the more commonly used *DatabaseMetaData* object methods:
 - **getDatabaseProductName()** Returns the product name of the database.
 - **getUserName()** Returns the username.
 - **getURL()** Returns the URL of the database
 - **getSchemas()** Returns all the schema names available in this database.
 - **getPrimaryKeys()** Returns primary keys.
 - **getProcedures()** Returns stored procedure names.
 - **getTables()** Returns names of tables in the database.

24. Explain the types of Exceptions occur in JDBC.

Exceptions

- There are 3 types of Exceptions that are thrown by JDBC Method. These are SQL Exception, SQL Warning and Data Truncation.
- *SQLExceptions* commonly reflect a SQL syntax error in the query and are thrown by many of the methods contained in the *java.sql* package. Hopefully, the syntax errors in your code get resolved quickly.
- In production, this exception is most commonly caused by connectivity issues with the database.
- It can also be caused by subtle coding errors like trying to access an object that's been closed.

- For example, you try to roll back a transaction in a catch clause and don't check first if the database connection is still valid.
- The **getNextException()** method of the **SQLExceptions** object is used to return details about the SQL error or a null if the last exception was retrieved.
- The **getErrorCode()** method of the **SQLException** object is used to retrieve vendor-specific error codes.
- The **SQLWarning** throws warnings received by the **Connection** from the DBMS.
- The **getWarnings()** method of the **Connection** object retrieves the warning and the **getNextWarning()** method of the **Connection** object retrieves subsequent warnings.
- Whenever data is lost due to truncation of the data value, a **DataTruncation** exception is thrown